



Technion – Israel Institute of Technology

Faculty of Electrical & Computer Engineering

VLSI Lab

Project A Final Report

Two Independent Tiny Tapeout ASIC Projects for ‘Achtung, die Kurve!’

Presented by: Amit Elmaliach & Joel Kaplan

Mentor: Roy Treves

Contents

1. Abstract	3
2. List of shorthands	4
3. Background and Motivation	5
3.1. Project Goals and Requirements	5
4. System Implementation.....	6
4.1. Top Level Architecture.....	6
4.2. PSRAM Controller (psram_controller.v).....	7
4.2.1. Timing Constraints and Data Packing	8
4.3. Game Logic FSM (game_fsm.v)	9
4.4. VGA Synchronization & Time Multiplexing (vga_sync.v)	10
4.5. Display Streamer (display_streamer.v).....	11
4.5.1. Mathematical Analysis of Optimal vs. Practical Buffer Sizing.....	11
4.5.2. HDL-Level Area Optimization	14
4.6. Independent 1x1 and 1x2 Projects	14
4.7. I/O Controller.....	15
5. Simulation and Verification Methodology.....	16
5.1. Python Behavioral Models and Co-Simulation	16
5.2. Deterministic Testing via JSON Traces	16
5.3. Automated Equivalence Checking.....	17
5.4. Subsystem and System-Level Verification	17
5.5. Per-Project Resolution Testing	17
5.6. Continuous Integration.....	18
6. Synthesis and Layout.....	19
7. Summary and Conclusions	21
8. Future work	22
9. References	23

1. Abstract

This report presents two independent hardware implementations of the multiplayer arcade game ‘Achtung, die Kurve!’ (also known as Curve Fever), prepared as separate ASIC projects for the Tiny Tapeout platform. The academic development and verification are discussed together, but the 1x1 and 1x2 designs are not combined into one Tiny Tapeout project: each has its own top module, source and configuration, hardening result, GDSII, repository, and portal submission. The main architectural challenge is real-time video generation within a very small standard-cell area. An external APS6404L-3SQR QSPI PSRAM stores the framebuffer: the 1x1 project uses a 1-bit occupancy representation, while the full 1x2 project stores 640x480 pixels at 2 bits per pixel.

A core feature of this research is the two-project implementation strategy, designed to explore the tradeoffs between silicon area and feature sets. The first standalone project is the highly constrained 1x1 ‘JA Achtung Compact,’ which aggressively optimizes logic to fit within one Tiny Tapeout tile at 92.6% utilization. The second is the standalone 1x2 ‘JA Achtung Full,’ which spans two tiles to support expanded gameplay mechanics and rich color presentation. They are developed and validated independently and will be published in two separate public repositories for two separate shuttle submissions.

Both independent projects accept two controllers through the Gamepad PMOD and generate 640x480 VGA timing. To reduce logic area, the 1x1 project uses a 160x120 logical movement grid expanded to the 640x480 output. Its packed occupancy map groups four horizontal pixels, and movement normally paints four physical rows per cell; player 2 uses a grooved variant in alternating cells. The 1x2 project operates directly on the full 640x480 framebuffer and adds 64-direction movement, selectable colors, boost, and passable trail gaps. Both implementations were checked separately with automated Python reference models and cocotb simulations.

The verification methodology combines deterministic JSON input traces, independent Python behavioral models, cocotb simulations of the Verilog RTL, subsystem tests, a complete top-level smoke test for each project, and separate gate-level simulations. The generated framebuffer maps are compared bit-for-bit, while waveform files remain available for debugging. These checks provide strong evidence of agreement for the tested scenarios before each project’s independent RTL hardening and GDSII generation.

2. List of shorthands

- TT - Tiny Tapeout
- ASIC - Application Specific Integrated Circuit
- FPGA - Field Programmable Gate Array
- RTL - Register Transfer Level
- PDK - Process Design Kit
- GDSII - Graphic Design System II (Standard database file format for data exchange of IC layout artwork)
- FSM - Finite State Machine
- VGA - Video Graphics Array
- I/O - Input/Output
- PSRAM - Pseudo Static Random-Access Memory
- QSPI - Quad Serial Peripheral Interface
- QPI - Quad Peripheral Interface (command, address, and data all use the four data lines)
- VCD - Value Change Dump
- COCOTB - Coroutine based co-simulation testbench
- CI - Continuous Integration
- PMOD - Peripheral Module (standard interface for small I/O boards)

3. Background and Motivation

Application Specific Integrated Circuits (ASICs) represent the pinnacle of digital design performance, but community driven platforms like Tiny Tapeout impose strict silicon area boundaries. A major limitation in small-scale ASIC design is memory capacity; storing a high-resolution video framebuffer strictly within internal standard cell flip-flops or local SRAM macros is physically impossible within a 1x1 or 1x2 tile constraint.

To overcome this, the project was motivated by the need to develop a robust external memory interface. 'Achtung die Kurve' is a game where players leave persistent trails, meaning the entire board state must be stored, read, and rendered continuously. By implementing a custom QSPI PSRAM controller, the project successfully offloads the video memory to an external chip. This architecture not only makes the game feasible but serves as a proof-of-concept for integrating complex external memory systems in area-constrained open-source silicon designs.

3.1. Project Goals and Requirements

The primary goal of this academic work was to demonstrate a streamlined and accessible approach to designing and manufacturing ASICs under external-memory constraints. The requirements applied separately to both Tiny Tapeout projects and are described below:

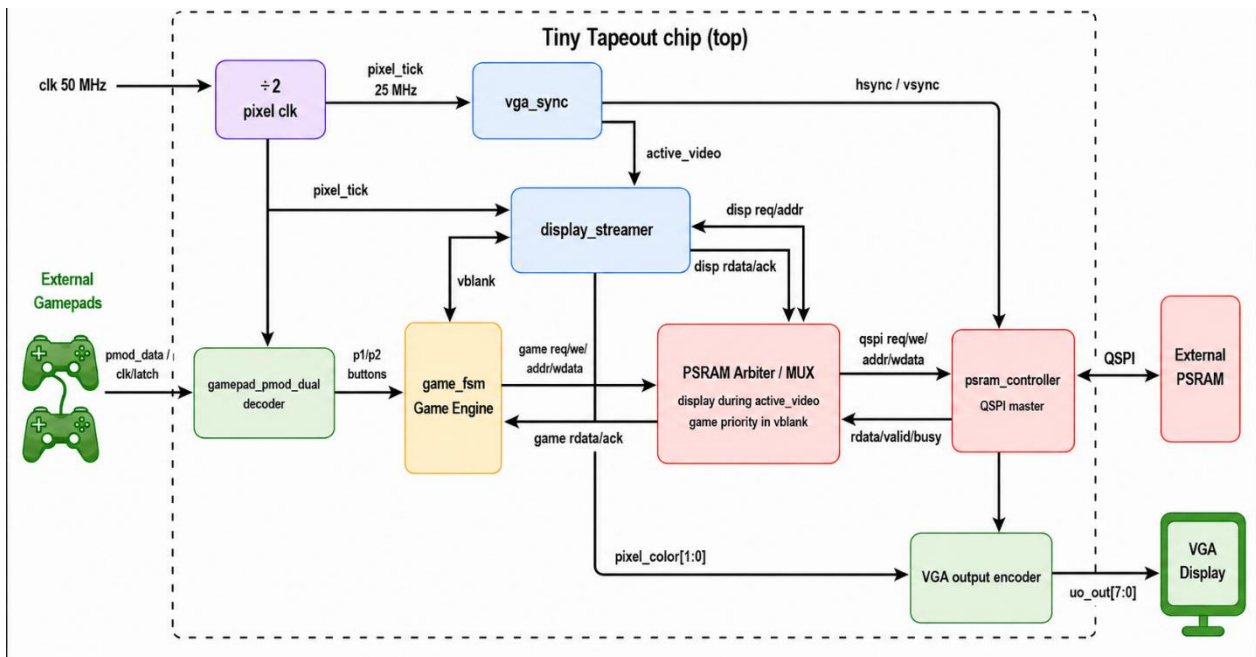
- Software prototyping of the game logic and PSRAM modeling.
- RTL design, simulation, and timing verification.
- Development of a custom QSPI PSRAM controller.
- Developing a protocol-level Python bus-functional model for the PSRAM interface.
- Implementing a low-latency I/O interface for two external player controllers.
- Safely synchronizing and decoding the asynchronous Gamepad PMOD signals, while applying gameplay updates and framebuffer accesses during the VGA vertical-blanking interval to avoid interference with active video.
- Two independent Tiny Tapeout project setups: a standalone 1x1 minimal project and a standalone 1x2 full-game project.
- Minimizing silicon die area while maintaining VGA timing compliance.
- Automated testing and verification utilizing JSON input traces.
- Generating and hardening a separate GDSII for each project using the TT flow.

4. System Implementation

The academic work began with Python behavioral models of both game variants. These models helped define the movement, collision, framebuffer-packing, and input semantics and later became independent references for verifying each project’s RTL.

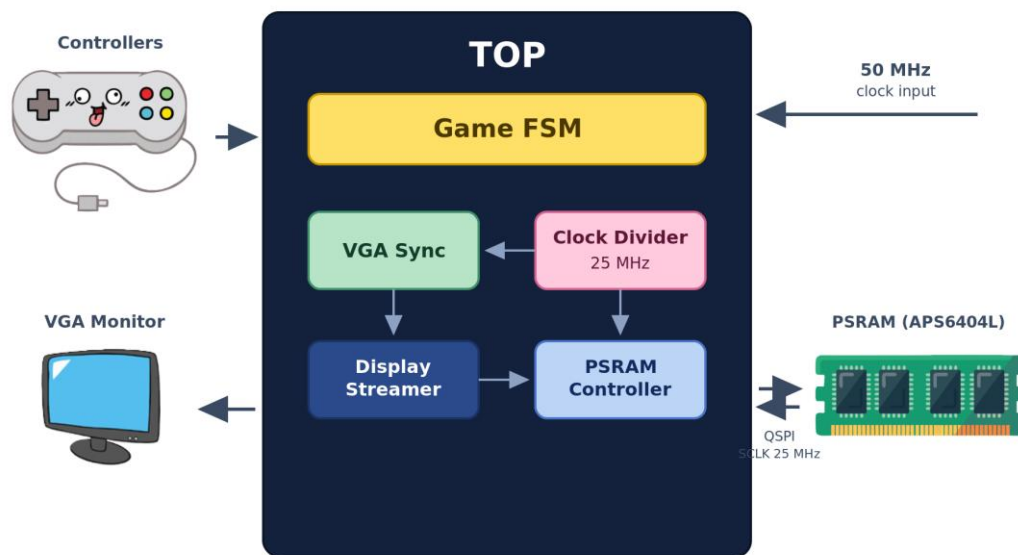
The final verification code is organized around `game_models.py`, which contains `Compact1x1Model` and `Curved2x1Model`, together with `maps.py` for the target-specific framebuffer layouts. An optional pygame front end in `live_record.py` provides interactive trace recording and visualization; pygame is not required for the automated comparison suite.

Deterministic JSON traces specify player-button states frame by frame. `target_runner.py` executes each trace in the selected Python model and in the corresponding Verilog simulation, writes comparable text maps, and reports any difference. The Python result is therefore a test oracle rather than an assumed proof of correctness.



4.1. Top Level Architecture

The two hardened top-level modules belong to separate Tiny Tapeout projects and are never instantiated together or combined into one GDSII. The 1x1 project uses `tt_um_ja_achtung_1x1` in `tt_um_ja_achtung_1x1.v`, while the 1x2 project uses `tt_um_ja_achtung_1x2` in `tt_um_ja_achtung_1x2.v`. Each independently coordinates VGA timing, framebuffer streaming, game state, controller input, and PSRAM arbitration. A 50 MHz Tiny Tapeout clock drives one synchronous clock domain; a clock-enable advances VGA at 25 MHz, and a registered divide-by-two output produces the 25 MHz PSRAM SCLK. The 50 MHz input is below Tiny Tapeout’s 66 MHz input limit, and the 25 MHz SCLK is below its approximately 33 MHz output-switching limit. The APS6404L supports 84 MHz linear bursts and higher rates in wrapped mode, so the external I/O limit—not the memory—sets the practical bound.



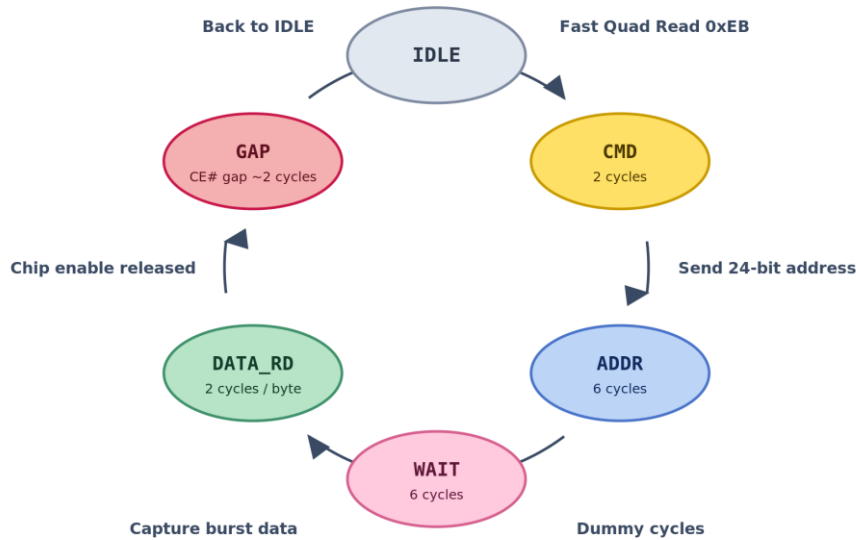
4.2. PSRAM Controller (psram_controller.v)

The `psram_controller.v` module connects the display and game clients to the external APS6404L-3SQR PSRAM through a Quad Serial Peripheral Interface (QSPI). Its command sequence, bus-direction changes, and wait cycles follow the device datasheet.

After the board has held reset low for the required power-up interval, the controller first emits a QPI-exit preamble so a warm device is returned to SPI mode. It then sends Reset Enable (0x66), Reset (0x99), and Enter QPI Mode (0x35). In QPI mode, command, address, and data phases use all four SIO lines.

For Fast Quad Read (0xEB), the controller enters `S_WAIT` for exactly six `SCLK` cycles before `S_DATA_RD` captures the returned nibbles. Display reads use fixed 4-byte bursts in the 1x1 target and fixed 8-byte bursts in the 1x2 target; the 1x1 game path uses single-byte reads and writes, while the 1x2 game path uses aligned 8-byte read-modify-write transactions. These short, aligned transactions remain well below the device's maximum `CE#`-low time without dynamic page-boundary tracking.

PSRAM Controller FSM



4.2.1. Timing Constraints and Data Packing

During architectural planning, the main memory-bandwidth constraint was identified. The APS6404L-3SQR is rated for 84 MHz linear bursts; its advertised 133 MHz rate applies to 32-byte wrapped bursts at $3.0\text{ V} \pm 10\%$. This design accepts a 50 MHz Tiny Tapeout clock and uses a 25 MHz clock-enable for VGA together with a registered 25 MHz PSRAM SCLK. Tiny Tapeout's approximately 33 MHz output-switching guidance therefore sets the practical SCLK bound. Because a visible pixel is consumed on each 25 MHz pixel-enable cycle while a PSRAM access also requires command, address, and wait phases, a direct one-access-per-pixel scheme cannot keep pace.

To overcome this bottleneck, each target combines packed framebuffer data with overlapped prefetching. The 1x2 target stores 2 bits per physical pixel, so one byte represents four 640x480 pixels. The 1x1 framebuffer stores one occupancy bit per four-pixel horizontal group on each of 480 physical rows. Its 160x120 movement engine normally paints four adjacent rows for a cell; player 2 omits three rows in alternating cells to form a distinguishing groove. One byte therefore covers 32 horizontal VGA pixels on one physical row.

In both cases, the display streamer fetches packed bursts ahead of the VGA beam and unpacks them on the fly. Packing amortizes transaction overhead across several display pixels, while overlapped prefetching lets the next read complete as the current burst is consumed. For the 1x2 target, the cycle model places the 8-byte burst at the estimated throughput boundary

rather than providing spare theoretical bandwidth; the full 640x480 VGA simulation verifies that the implemented pipeline keeps pace for the tested pattern.

4.3. Game Logic FSM (game_fsm.v)

The two targets contain different game_fsm.v implementations. Both calculate the next position, reject wall crossings, read the target framebuffer location before committing a trail write, and stop the round after a collision. The table below summarizes the compact 1x1 cardinal-movement FSM. The 1x2 engine additionally implements a color-selection lobby, Q10.6 fixed-point positions, 64 movement directions, boost moves, distance-based trail gaps, diagonal side-pixel filling, and burst-based framebuffer clearing.

Compact 1x1 game FSM:

Current State	Primary Function / Actions	Transition Condition	Next State
S_CLEAR (Power On)	Sequentially clears the entire PSRAM framebuffer byte-by-byte before gameplay begins. Resets player coordinates.	clear_byte != FRAME_BYTES_COUNT && game_ack (Memory write acknowledged)	S_CLEAR
		clear_byte == FRAME_BYTES_COUNT (Clearing complete)	IDLE
IDLE	Waits for the hardware to signal the start of a new VGA frame.	frame_start is high && paint_row == 3	ANGLE
		No frame_start detected	IDLE
ANGLE	Samples the already-decoded button state for the active player and performs edge detection for discrete 90-degree turns.	vblank is active	CHECK
		!vblank (Failsafe timeout back to wait phase)	IDLE
CHECK	Calculates the player's next logical X/Y coordinate based on current direction.	vblank && hit_wall (Boundary collision detected)	S_OVER

	Evaluates if the next step hits the outer walls.		
		vblank && !hit_wall (Valid path step)	READ
		!vblank (Failsafe timeout)	IDLE
READ	Asserts a memory read request to fetch the PSRAM pixel data at the player's calculated next coordinate.	vblank && game_ack && target_pixel_data == 1 (Trail collision detected)	S_OVER
		vblank && game_ack && target_pixel_data == 0 (Path is clear)	WRITE
		!vblank (Failsafe timeout)	IDLE
WRITE	Writes the new player trail pixel data to PSRAM. Uses a loop to paint 4 physical rows (paint_row) per logical coordinate.	vblank && game_ack && paint_row != 3 (Row loop incomplete)	READ
		vblank && game_ack && paint_row == 3 && !active_player (Player 1 turn complete)	ANGLE
		vblank && game_ack && paint_row == 3 && active_player (Player 2 turn complete)	IDLE
		!vblank (Failsafe timeout)	IDLE
S_OVER	Game Over state. Halts all logical updates and waits for user input.	restart signal triggered by gamepad controller	S_CLEAR

4.4. VGA Synchronization & Time Multiplexing (vga_sync.v)

This module generates the necessary HSYNC and VSYNC timing signals for standard 640x480 VGA output. It maintains pixel counters to track screen coordinates and generates a vertical blanking (vblank) signal. The game FSM execution is seamlessly tied to this vblank signal, ensuring game logic updates without screen tearing.

Because both the VGA output and the internal Game FSM require access to the single external PSRAM chip, an arbitration mechanism was required to prevent bus collisions and

screen tearing. The system handles this using strict Time Division Multiplexing (TDM) tied directly to the VGA synchronization signals.

The memory access is divided into two distinct phases:

1. **Active Video Phase:** During the visible portion of the screen rendering, the Display Streamer is granted absolute priority. It continuously requests data bursts from the PSRAM to ensure the monitor is constantly fed with pixel data. The Game FSM is stalled and cannot access memory during this time.
2. **Vertical Blanking (VBLANK) Phase:** When the visible frame ends, vblank goes high and the Game FSM receives priority for PSRAM transactions. Visible-pixel consumption has stopped, but the display path may use otherwise idle blanking cycles to prefetch when no game request is pending.

During vertical blanking, the Game FSM reads target pixels for collision detection and writes new trail pixels to PSRAM. If no game request is pending, the display side may use remaining blanking cycles to prefetch the first burst of the next frame. With a 25 MHz pixel enable and the standard 800x525 timing totals, the implemented refresh rate is approximately 59.5 frames per second; the pixel-exact VGA test verifies continuous output for the tested pattern.

4.5. Display Streamer (display_streamer.v)

Operating in tandem with the VGA sync, the display streamer continuously reads packed pixels from the PSRAM memory map. Because memory reads take multiple cycles, the display path overlaps prefetching with pixel consumption without instantiating two standalone local display buffers. One local shift register holds the active burst, while the PSRAM controller's existing read-data register temporarily holds the prefetched burst. Packed pixels are unpacked on the fly to drive the color output pins in real time.

4.5.1. Mathematical Analysis of Optimal vs. Practical Buffer Sizing

The fundamental challenge of real time VGA generation is ensuring the memory fetch rate outpaces the display drain rate without consuming excessive standard cell area. The analysis below is carried out for the more demanding case — the 1x2 full-game architecture at 2 bits per pixel (2bpp) — and then revisited for the 1x1. The system operates on a 50 MHz system clock; both the VGA pixel tick and the PSRAM QPI clock (SCLK) run at 25 MHz, and all cycle counts below are in 25 MHz SCLK cycles. At 2bpp, a single byte (8 bits) of memory represents exactly 4 physical VGA pixels.

Let N represent the buffer size in bytes.

The time required to display the buffered pixels, measured in 25 MHz pixel clock cycles, is directly proportional to the packing ratio:

$$C_{disp} = 4N$$

To fetch this data, the PSRAM controller must execute a QPI Fast Quad Read (0xEB) sequence. In QPI mode (4 bits per cycle), the transaction overhead in 25 MHz cycles includes:

- **Command (0xEB, 8 bits):** 2 cycles
- **24-bit Address:** 6 cycles
- **Wait States:** 6 cycles
- **Overhead (CE# gap & state transitions):** ~2 cycles
- **Data Transfer: 2 cycles per byte = 2N cycles**

The total fetch time in 25 MHz clock cycles is therefore:

$$C_{fetch} = 16 + 2N$$

To prevent screen tearing and pixel starvation, the display time must be strictly greater than or equal to the memory fetch time:

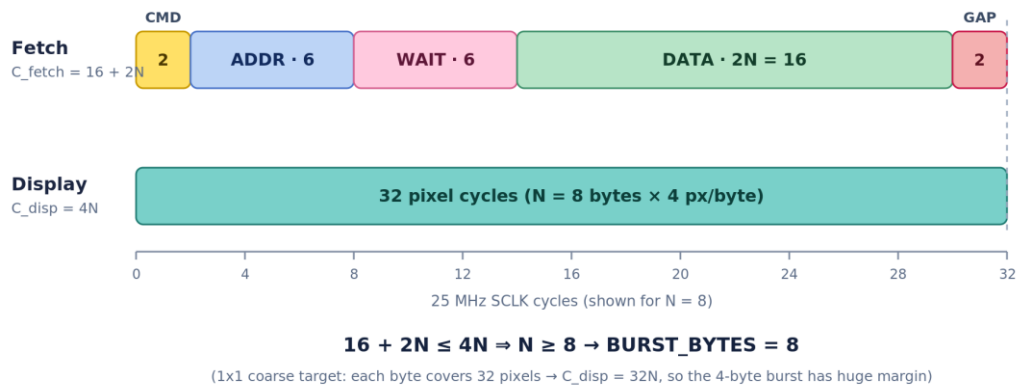
$$C_{fetch} \leq C_{disp}$$

$$16 + 2N \leq 4N$$

$$N \geq 8$$

This implementation-specific cycle model explains why small bursts cannot sustain the full-resolution 2bpp stream. Under the stated command, address, wait, and inter-transaction overhead assumptions, fetching one byte would require about 18 SCLK cycles while displaying it would take only four pixel cycles.

An 8-byte (64-bit) burst is the minimum predicted by this cycle model: its estimated 32-cycle fetch time matches the 32 pixel cycles over which its data is displayed. The full 640x480 VGA simulation provides the implementation-level check that this boundary case operates correctly.



Buffer Sizing & Resource Tradeoff Analysis

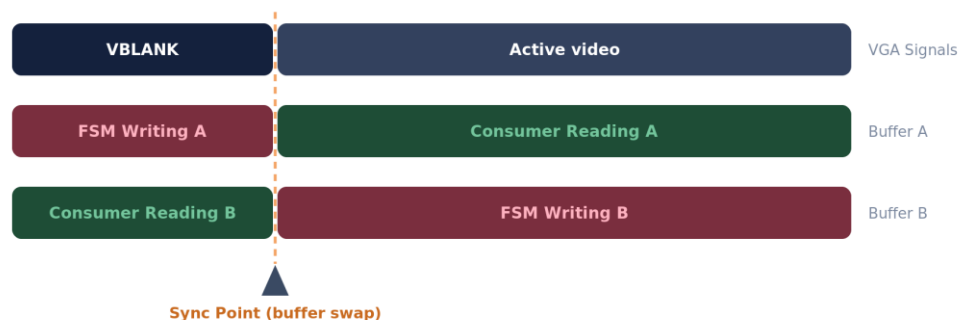
Buffer Size	Display Time (C_{disp})	Fetch Time (C_{fetch})	Status	Naive Two-Buffer Storage Cost
1 Byte (8-bit)	4 cycles	18 cycles	FAIL (severe data starvation)	16 FFs (extremely low)
4 Bytes (32-bit)	16 cycles	24 cycles	FAIL (display still outruns fetch)	64 FFs
8 Bytes (64-bit)	32 cycles	32 cycles	PASS (absolute minimum threshold)	128 FFs (optimal / chosen)
16 Bytes (128-bit)	64 cycles	48 cycles	PASS (large margin / idle bus)	256 FFs (unnecessary area)

Conclusion on Prefetch Register Reuse:

While an even larger buffer would make the display still less sensitive to short-term RAM latency, it demands significant silicon area that threatens the Tiny Tapeout utilization limits.

The 1x2 therefore uses an 8-byte burst with overlapped prefetching. `display_shift` holds the burst currently being consumed, while the PSRAM controller's `o_rdata` register temporarily retains the next completed read. Reusing that existing read-data register avoids adding a second standalone 64-bit display buffer while still hiding the protocol overhead.

The 1x1 case: in the compact target the packing is far denser—each byte holds occupancy bits for eight horizontal logical-cell columns on one physical row. Each column spans four horizontal VGA pixels, so one byte covers 32 horizontal pixels and $C_{\text{disp}} = 32N$. With $C_{\text{fetch}} = 16 + 2N$, even a single-byte buffer would satisfy the rate condition ($18 \leq 32$). The 1x1 therefore uses a comfortable 4-byte burst (`BURST_BYTES = 4`): its fetch takes 24 cycles against 128 cycles of display time, leaving substantial bandwidth margin.



4.5.2. HDL-Level Area Optimization

Framebuffer packing is only one part of the area reduction. Both designs were also optimized at RTL level through resource sharing, datapath reuse, register-width reduction, and elimination of redundant storage. These changes do not use general-purpose compression; instead, hardware that is not needed simultaneously is time-multiplexed so that one circuit performs several related operations.

In each game FSM, an active-player selector routes both players through one movement, wall/trail-collision, framebuffer-address, and PSRAM transaction datapath rather than duplicating those blocks. The 1x2 additionally uses one 16-entry first-quadrant angle table and mirrors, swaps, and signs its outputs to generate all 64 directions. Row-address multiplication was replaced by shift-and-add logic, internal widths were narrowed, and stable intermediate values are derived when needed rather than stored in additional registers.

The display path was reduced from two standalone local burst buffers to one active display_shift register. The existing PSRAM-controller o_rdata register temporarily holds the prefetched next burst, so it also serves as the second storage stage without another display-buffer copy; in the 1x2 controller, o_rdata is additionally reused as the write shifter. The wide display register is not reset directly and is instead protected by a reset valid bit, avoiding reset circuitry on every data bit. A small arbiter also lets the game and display clients share one PSRAM controller.

The 1x1 applies further reductions: a 1-bit occupancy map, 4x4 logical movement cells, byte-wide game updates, narrower coordinates, and a minimal controller decoder that stores only the required inputs. Together, these transformations reduced flip-flop and combinational area while maintaining the behavior checked by the regression, full-system, and gate-level tests.

4.6. Independent 1x1 and 1x2 Projects

To evaluate layout efficiency, two standalone Tiny Tapeout projects were implemented. During development, they are maintained as standalone roots on the separate tt-1x1 and tt-1x2 branches of a private staging repository; checking out either branch yields one complete Tiny Tapeout project, never both. For shuttle submission, each branch root becomes its own public repository, workflow, GDSII, TTSKY26c portal project, and submission. They share architectural ideas and verification utilities, but neither submitted project contains or instantiates the other design:

1x1 ‘JA Achtung Compact’ (tt_um_ja_achtung_1x1): This single-tile version uses orthogonal

movement, monochrome occupancy data, and a compact Gamepad PMOD decoder. It generates 640x480 VGA while the game advances on a 160x120 logical grid. Stored occupancy bits expand across four horizontal pixels, and movement normally paints four physical rows per cell; player 2 uses a grooved variant for visual distinction. Its display streamer uses 4-byte (32-bit) bursts.

1x2 ‘JA Achtung Full’ (tt_um_ja_achtung_1x2): The two-tile version operates directly on the 640x480, 2bpp framebuffer and adds selectable colors, a lobby, 64-direction fixed-point movement, boost, 32-pixel passable trail gaps, diagonal trail filling, and a 2x2-pixel live head marker while a gap is being drawn. Its display streamer uses 8-byte (64-bit) bursts, and the final layout passes the routing and physical checks listed in Section 6.

4.7. I/O Controller

This module manages the communication with the external player controllers. The physical input hardware is the Gamepad PMOD, which carries two SNES-style controllers over a single shared serial interface (latch, clock, and data). The two targets take different approaches to decoding it:

- The 1x2 target integrates Pat Deegan’s Gamepad PMOD interface module (Apache-2.0 licensed; attribution is retained in the source), which decodes the full button set of both controllers.
- The 1x1 target, where every standard cell counts, uses our own compact serial decoder that extracts only the buttons the game actually needs; it is verified by an exhaustive 8193-case test (see Section 5).

In both independent projects, the asynchronous PMOD data, clock, and latch signals pass through two-stage synchronizers before the serial decoders consume them. Decoding occurs continuously as PMOD frames arrive; the Game FSM applies movement inputs and performs framebuffer accesses during vertical blanking. This separates metastability protection from the video-time arbitration policy.



Physical Hardware Components

5. Simulation and Verification Methodology

Verifying either ASIC project as it interacts with external memory and produces real-time VGA requires more than manual waveform inspection. The shared verification framework therefore runs automated cocotb simulations with Icarus Verilog, deterministic traces, reference models, protocol-level bus-functional models, and dedicated subsystem tests separately for each project.

The core method is differential checking: the Python and RTL models execute the same deterministic inputs, and their final framebuffer maps are compared bit-for-bit. This is strong functional testing of the covered scenarios, although it is not formal equivalence verification.

5.1. Python Behavioral Models and Co-Simulation

To establish a baseline source of truth, two critical behavioral models were developed in Python:

1. **The PSRAM bus-functional model (`psram_bfm.py`):** SimplePsram provides an 8 MiB byte-addressable memory image, and SplitPsramBFM observes the real QSPI pins. It checks SPI/QPI initialization, command and address widths, bus direction, six read wait cycles, fixed-size data transfers, and top-level pin mapping. It intentionally does not claim electrical timing, refresh, tCEM enforcement, or wrapped-page behavior beyond what the controller uses.
2. **The game reference models (`game_models.py`):** Compact1x1Model and Curved2x1Model independently implement the two movement engines and use `maps.py` for target-specific framebuffer packing. `target_runner.py` applies JSON traces to these models and to the corresponding Verilog testbenches.

5.2. Deterministic Testing via JSON Traces

Testing interactive gameplay requires repeatable inputs. `trace_gen.py` deterministically generates the committed JSON traces, and CI regenerates them and checks that no file changes. Each trace records frame-indexed controller states and expected collision outcomes where applicable.

The generator produces 21 scenarios for the 1x1 target in both full 640x480 and accelerated 64x48 modes, plus 23 full-resolution and 24 coarse traces for the 1x2 target. The tests are categorized by complexity:

- **Easy & Medium Tests:** Basic movements, 90-degree turns, self-crashes, and boundary collisions.
- **Stress Tests:** Holding buttons, simultaneous opposite inputs, and high-frequency "staircase" maneuvers.

- **Seeded Fuzzing:** Deterministic randomized traces cover sparse, active, and chaotic two-player input patterns. Coarse traces run for up to 800 frames, while the full-resolution timelines scale to as many as 8,000 frames. Completion and final-map agreement help expose deadlocks or state-machine divergence under unpredictable input sequences.

These JSON traces are ingested identically by both the Python simulator and the Verilog cocotb testbench, guaranteeing that both environments execute the exact same gameplay scenario.

5.3. Automated Equivalence Checking

For each game trace, the Python model and the direct-RAM Verilog testbench write structured text maps of the target framebuffer. `target_runner.py` compares these maps and fails the test on the first mismatch. A separate full-top smoke test routes controller data through the Gamepad PMOD decoder and every memory access through the real QSPI controller before comparing the resulting PSRAM image.

`maps.py` contains the 1-bit 1x1 and 2bpp 1x2 packing and text-map functions. The direct game tests use behavioral memory arrays for speed, while `cocotb_system_compare.py` uses `SplitPsrAmBFM` and the complete Tiny Tapeout top. Agreement means the tested RTL behavior matches the independent model for that trace; any discrepancy is reported as a functional failure.

5.4. Subsystem and System-Level Verification

Beyond the game-trace suites, several dedicated tests exercise the remaining hardware. `cocotb_vga_picture.py` captures and checks a pixel-exact 640x480 VGA frame through the real QSPI read path and display streamer.

The VGA test preloads a known PSRAM pattern, observes `hsync`, `vsync`, and RGB, and compares the reconstructed frame pixel-for-pixel. Dedicated RAM tests cover `psram_controller.v` and `display_streamer.v`; `cocotb_system_compare.py` exercises gamepad input, QSPI traffic, and display/game arbitration together at accelerated geometry; and `tb_decoder_exhaustive.sv` checks 8,193 compact-decoder cases.

5.5. Per-Project Resolution Testing

The shared verification framework runs separately against both standalone projects:

- **Accelerated mode (`--mode coarse`):** runs either target's game logic at 64x48 to shorten simulation time.

- **Full mode (--mode full):** runs either target's game logic at its real 640x480 geometry. The separate --full-vga option selects the complete 640x480 VGA capture test.

Each project has its own run.py entry point invoking target_runner.py. Its CI independently runs the full and coarse game suites, a true 640x480 VGA capture, the accelerated full-system smoke test, PSRAM/display unit tests, root RTL tests, GDS hardening, Tiny Tapeout precheck, and a powered gate-level smoke test.

5.6. Continuous Integration

Each standalone branch has its own GitHub Actions workflows and runs its own RTL tests, game and RAM suites, GDS hardening, Tiny Tapeout precheck, and gate-level simulation. A failing functional or physical check prevents that project commit from becoming its independent submission candidate.

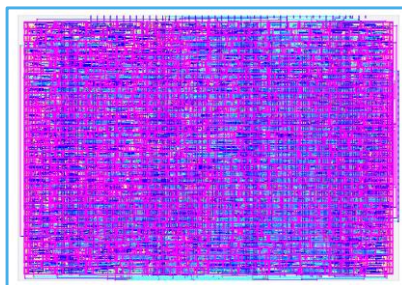
6. Synthesis and Layout

Synthesis and physical design use the open-source LibreLane flow provided by Tiny Tapeout (LibreLane 3.0.3, sky130A PDK revision 8afc8346, and a 20 ns clock constraint). Each standalone project has its own validated hardening baseline intended for the TTSKY26c shuttle. The results below describe the two independently green branch heads; neither is yet a portal submission.

Validated staging hardening results for the two independent planned TTSKY26c submissions:

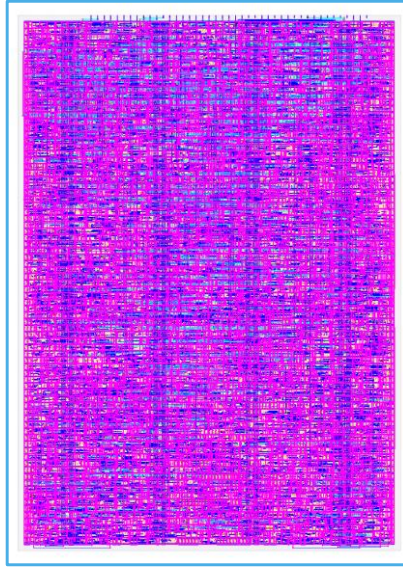
Metric	1x1 “JA Achtung Compact”	1x2 “JA Achtung Full”
Top module	tt_um_ja_achtung_1x1	tt_um_ja_achtung_1x2
Tile size	1x1 (161.00 × 111.52 μm)	1x2 (161.00 × 225.76 μm)
Standard cell area	15,279.7 μm ²	31,142.4 μm ²
Utilization	92.64%	90.91%
Worst setup slack	+5.224 ns (max_ss_100C_1v60)	+0.577 ns (max_ss_100C_1v60)
Worst hold slack	+34.4 ps (min_ff_n40C_1v95)	+33.6 ps (min_ff_n40C_1v95)
Peak max-slew advisory count in any one corner	174 (19 at nominal TT)	440 (22 at nominal TT)
Peak max-cap / max-fanout advisory count in any one corner	0 / 0	1 / 0
Hard violations (DRC/LVS/antenna/latches/power grid)	0	0
Tiny Tapeout precheck	15/15 passed	15/15 passed
Gate-level test	Passed	Passed

- **1x1 ‘JA Achtung Compact’ (tt_um_ja_achtung_1x1):** The final baseline uses 15,279.7 μm² of standard-cell area at 92.64% utilization and retains +5.224 ns of worst-corner setup slack. Routing DRC, Magic DRC, LVS, antenna, latch, and power-grid checks report no hard violations. Setup and hold slack are positive at all nine analyzed corner combinations.



COMPACT VERSION GDS VIEW

- **1x2 ‘JA Achtung Full’ (tt_um_ja_achtung_1x2):** The final baseline uses 31,142.4 μm^2 of standard-cell area at 90.91% utilization. Its worst-corner setup slack is +0.577 ns, and setup and hold remain positive at all nine analyzed corner combinations. The same routing and physical signoff checks report no hard violations.



FULL VERSION GDS VIEW

A note on hold margins: fitting both dense targets required setting the placement and global-route hold-fixing margins to 0.02 ns; larger margins introduced enough additional repair area to prevent legalization. The minimum reported hold slack is +34.4 ps for the 1x1 and +33.6 ps for the 1x2 at the fast -40°C, 1.95 V corner, while the 25°C typical-PVT combinations remain near +0.21 ns. This is a deliberate area-versus-margin trade-off, not a guarantee of silicon behavior. LibreLane nevertheless reports non-negative setup and hold slack at every analyzed corner. The max-slew figures in the table are worst-corner advisory counts—174 for the 1x1 and 440 for the 1x2—not sums across corners; the nominal-RC TT counts are 19 and 22. The 1x2 also has one worst-corner max-capacitance advisory.

Both branch heads pass the complete Tiny Tapeout precheck (15/15) and powered gate-level simulation of their hardened netlists. Each branch produces its own GDSII and Tiny Tapeout submission artifact. Final readiness still requires publishing two separate standalone repositories, enabling GitHub Pages through Actions in each repository, confirming both complete public-repository workflows including their viewers, and submitting each repository URL as a separate Tiny Tapeout portal project and revision.

7. Summary and Conclusions

A major factor in the two-project development cycle was the shared Python verification environment. Independent game models, deterministic traces, and reusable map utilities made it possible to debug and retest each Verilog implementation before its separate physical-design flow.

The resulting architecture combines an external PSRAM framebuffer, two-controller Gamepad PMOD input, and real-time VGA generation in a small Tiny Tapeout footprint. The integration required explicit scheduling of display and game memory traffic, target-specific data packing, and careful area optimization.

The custom QSPI PSRAM controller removes the need to implement the framebuffer in standard-cell storage and makes the 640x480 output practical within the available tile areas.

The two-project strategy quantifies the trade-off between area and features. The compact 1x1 project reaches 92.6% utilization with cardinal movement and a 1-bit occupancy map, while the independent 1x2 project reaches 90.9% utilization with full-resolution 2bpp storage, selectable colors, boost, gaps, and angled trajectories. They remain two separate Tiny Tapeout submissions rather than configurations combined into a single submitted project.

Automated Python-to-RTL comparisons, JSON traces, subsystem tests, physical checks, and gate-level simulation substantially increase confidence in the submission candidates. Physical silicon bring-up remains necessary to validate the I/O timing, PSRAM interface, VGA output, and gameplay under real electrical conditions.

8. Future work

This academic work and its two independent ASIC projects provide a strong foundation for future hardware-game development on open-source platforms. Future expansions could include:

1. **Silicon Bring-Up and Testing:** the 1x1 and 1x2 projects are planned as two separate submissions to the open TTSKY26c shuttle, each from its own public repository and portal project. The submission deadline is September 7, 2026. Tiny Tapeout currently lists March 27, 2027 as the expected chip date and May 12, 2027 as the estimated delivery date. After delivery, bring-up will validate the QSPI PSRAM link, VGA output, controller interface, and gameplay on physical silicon.
2. **Audio and Analog Enhancements:** Tiny Tapeout offers analog pins. Future iterations could use these to implement analog inputs (like a true joystick instead of digital buttons) or to output raw audio signals to a speaker based on collision events.
3. **Advanced Memory Topologies:** Utilizing the PSRAM interface to store not only the game trails but external graphical assets, sprites, and fonts, allowing for a fully stylized user interface

9. References

1. Tiny Tapeout: Hardening Tiny Tapeout Projects Locally, <https://tinytapeout.com/guides/local-hardening/>
2. Tiny Tapeout: Working with HDLs, <https://tinytapeout.com/hdl/>
3. Tiny Tapeout: Tech Specs, <https://tinytapeout.com/specs/>
4. Cocotb Documentation: <https://docs.cocotb.org/en/stable/>
5. AP Memory, APS6404L-3SQR QSPI PSRAM Datasheet, <https://www.apmemory.com/en/downloadFiles/032411212009597427>
6. J. Kaplan and A. Elmaliach, shared private staging repository, <https://github.com/Amite1000/Project1> (the standalone tt-1x1 and tt-1x2 branch roots will be published as separate public repositories and submitted independently).
7. P. Deegan, Gamepad PMOD interface module (Apache-2.0), <https://github.com/psychogenic/gamepad-pmod>